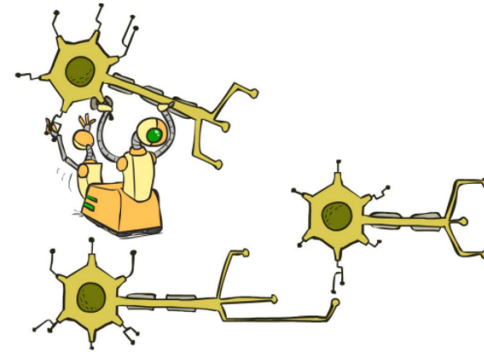


# SI3003 - Inteligencia Artificial

CLASE 7 – REGRESIÓN LINEAL, CLASIFICADORES LINEALES Y REDES NEURONALES

# Hoy

- De los datos al modelo: notación y función de costo
- **Regresión Lineal**
  - El modelo y su forma vectorial
  - El error y la función objetivo
  - Entrenamiento: gradiente y ecuación normal
- **Clasificadores Lineales**
  - El perceptrón de umbral
  - Regla de decisión binaria e hiperplano
  - Regla de aprendizaje y convergencia
- **Redes Neuronales y DNN**
  - De la regresión logística a la neurona
  - Sigmoide, softmax y multiclase
  - Arquitectura profunda y activaciones
  - Backpropagation
  - Aproximadores universales



# Parte 1 — Regresión Lineal

# Notación matemática

Antes del modelo, fijemos el lenguaje. Un conjunto de datos es una tabla: cada **fila** es un ejemplo y cada **columna** es una característica (*feature*).

|   | longitude | latitude | housing_median_age | total_rooms | total_bedrooms | population | households | median_income | median_house_value | ocean_proximity |
|---|-----------|----------|--------------------|-------------|----------------|------------|------------|---------------|--------------------|-----------------|
| 0 | -122.23   | 37.88    | 41.0               | 880.0       | 129.0          | 322.0      | 126.0      | 8.3252        | 452600.0           | NEAR BAY        |
| 1 | -122.22   | 37.86    | 21.0               | 7099.0      | 1106.0         | 2401.0     | 1138.0     | 8.3014        | 358500.0           | NEAR BAY        |
| 2 | -122.24   | 37.85    | 52.0               | 1467.0      | 190.0          | 496.0      | 177.0      | 7.2574        | 352100.0           | NEAR BAY        |
| 3 | -122.25   | 37.85    | 52.0               | 1274.0      | 235.0          | 558.0      | 219.0      | 5.6431        | 341300.0           | NEAR BAY        |
| 4 | -122.25   | 37.85    | 52.0               | 1627.0      | 280.0          | 565.0      | 259.0      | 3.8462        | 342200.0           | NEAR BAY        |

Vector de características

$$\mathbf{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix} \in \mathbb{R}^{d \times 1}$$

$y^{(i)} \in \mathbb{R}$   
 $y^{(i)} \in [0, 1, \dots, C]$

Matriz de características

$$\mathbf{X} = \begin{bmatrix} \dots & \mathbf{x}^{(1)\top} & \dots \\ \dots & \mathbf{x}^{(2)\top} & \dots \\ & \vdots & \\ \dots & \mathbf{x}^{(n)\top} & \dots \end{bmatrix} \in \mathbb{R}^{n \times d} \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

- $\mathbf{x}^{(i)} \in \mathbb{R}^{d \times 1}$ : el **vector de características** del ejemplo  $i$  (sus  $d$  columnas).
- $\mathbf{X} \in \mathbb{R}^{n \times d}$ : la **matriz de características**, con los  $n$  ejemplos apilados por filas.
- $y^{(i)}$ : la **etiqueta**. Si  $y^{(i)} \in \mathbb{R}$  el problema es de *regresión*; si  $y^{(i)} \in [0, 1, \dots, C]$  es de *clasificación*.

# Métodos de aprendizaje de máquina

Todo método de aprendizaje supervisado se reduce a dos objetos:

- Una **hipótesis**  $f$  que aproxima la relación entre  $\mathbf{x}^{(i)}$  y  $y^{(i)}$ .
- Una **función de costo**  $\mathcal{L}$  que mide qué tan mal lo está haciendo.

Formas de aproximar la relación entre  $\mathbf{x}^{(i)}$  y  $y^{(i)}$

$$f(\mathbf{x}^{(i)}) = y^{(i)}$$

Función de costo

$$\mathcal{L}(f(\mathbf{x}^{(i)}), y^{(i)})$$

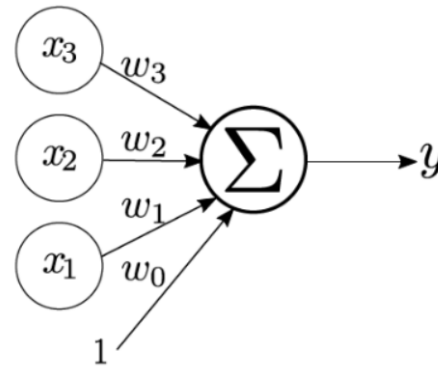
**Aprender** = elegir los parámetros de  $f$  que minimizan  $\mathcal{L}$  sobre los datos de entrenamiento. Todo lo que veremos hoy cambia *quién es  $f$*  y *quién es  $\mathcal{L}$* , pero nunca cambia esta receta.

# Regresión Lineal: Modelo

La hipótesis más simple posible: una **combinación lineal** de las características, más un término independiente  $w_0$  (el *sesgo* o *bias*).

$$\hat{y} = \hat{f}(\mathbf{x}) = w_0 + \sum_{j=1}^d w_j \times x_j$$

$$\hat{y} = \mathbf{w}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$

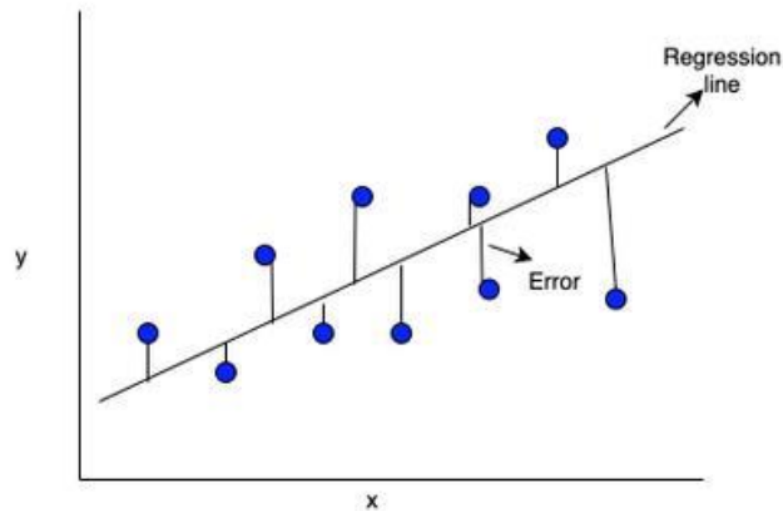


Absorbiendo el sesgo dentro del vector de pesos —agregando una entrada constante  $1$  al vector de características— el modelo se escribe como un solo producto punto.

Fíjense en el diagrama de la derecha: entradas, pesos, una suma y una salida. Eso **ya es una neurona artificial**. La regresión lineal es una red neuronal de una sola unidad, sin función de activación.

# Regresión Lineal: Error en la función objetivo

Para cada ejemplo, el **error** (o *residuo*) es la distancia vertical entre el valor real  $y_i$  y el valor predicho  $\hat{y}_i$  por la recta.



Ningún conjunto de pesos elimina todos los errores a la vez. Lo que sí podemos hacer es buscar la recta que los deja

# Regresión Lineal: Función Objetivo

Sumamos los errores **al cuadrado** sobre los  $n$  ejemplos y promediamos. Ese es el *error cuadrático medio*:

$$Cost_{\mathbf{w}} = \frac{1}{m} \sum_{i=1}^n \left( y_i - \mathbf{w}^T \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix} \right)^2$$

Entrenar la regresión lineal es

**minimizar esta función respecto a  $\mathbf{w}$**

. Es un problema de optimización convexo: tiene un único mínimo global, sin mínimos locales donde quedarse atrapado.



# Regresión Lineal: Entrenamiento

Hay dos caminos para llegar al mismo  $\mathbf{w}$  óptimo:

Descenso por el gradiente:

$$\mathbf{w} := \mathbf{w} - \alpha \times \nabla Cost_{\mathbf{w}}$$
$$\nabla Cost_{\mathbf{w}} = \frac{1}{m} \mathbf{X}^T (\hat{y} - y)$$

Ecuación normal:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} (\mathbf{X}^T \mathbf{Y})$$

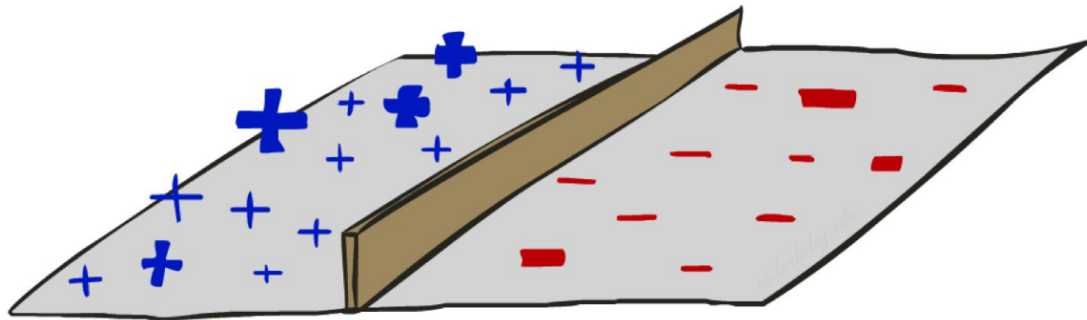
- **Descenso por el gradiente:** iterativo. Damos pasos de tamaño  $\alpha$  (la *tasa de aprendizaje*) en la dirección contraria al gradiente, hasta converger. Escala bien a muchos datos y a muchas dimensiones.
- **Ecuación normal:** solución cerrada. Un solo cálculo da la respuesta exacta, pero requiere invertir  $\mathbf{X}^T \mathbf{X}$  — costoso cuando  $d$  es grande.

En redes neuronales la ecuación normal no existe, pero el gradiente sí.

# Parte 2 — Clasificadores Lineales

# El perceptrón de umbral como clasificador lineal

¿Y si en vez de predecir un número queremos predecir una **clase**? La idea se conserva casi entera: seguimos calculando  $\mathbf{w} \cdot \mathbf{x}$ , pero ahora, en vez de reportar ese número, lo comparamos contra un **umbral**.



El resultado es una **frontera de decisión** que parte el espacio en dos: de un lado los ejemplos positivos, del otro los negativos.

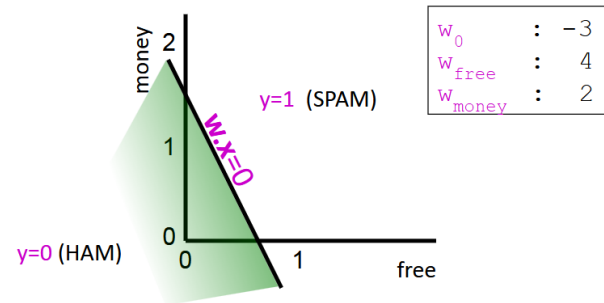
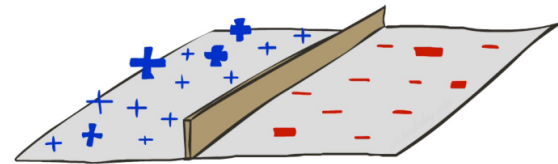
# Regla de decisión binaria

Un **perceptrón de umbral** es una sola unidad que produce:

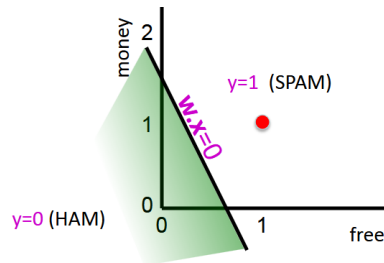
$$y = h_{\mathbf{w}}(\mathbf{x}) = \begin{cases} 1 & \text{si } \mathbf{w} \cdot \mathbf{x} \geq 0 \\ 0 & \text{si } \mathbf{w} \cdot \mathbf{x} < 0 \end{cases}$$

En el espacio vectorial de entrada:

- Los ejemplos son **puntos  $\mathbf{x}$** .
- La ecuación  $\mathbf{w} \cdot \mathbf{x} = 0$  define un **hiperplano**.
- Un lado corresponde a  $y = 1$ .
- El otro lado corresponde a  $y = 0$ .



# Ejemplo



|                    |      |
|--------------------|------|
| $w_0$              | : -3 |
| $w_{\text{free}}$  | : 4  |
| $w_{\text{money}}$ | : 2  |

|                    |     |
|--------------------|-----|
| $x_0$              | : 1 |
| $x_{\text{free}}$  | : 1 |
| $x_{\text{money}}$ | : 1 |

Dear Stuart, I'm leaving Microsoft to return to academia. The **money** is is great here but I prefer to be **free** to do my own research; and I really love teaching undergrads!

Do I need to finish my BA first before applying?

Best wishes

Bill

$$\mathbf{w} \cdot \mathbf{x} = -3x_1 + 4x_1 + 2x_1 = 3$$

El correo contiene *free* y *money*, así que  $\mathbf{x} = (1, 1, 1)$ . Con  $\mathbf{w} = (-3, 4, 2)$ :  $\mathbf{w} \cdot \mathbf{x} = -3 + 4 + 2 = 3 \geq 0 \Rightarrow y = 1$  (SPAM).

# Regla de aprendizaje del perceptrón

Si la etiqueta real  $y$  difiere de la predicción  $h_{\mathbf{w}}(\mathbf{x})$ , hubo un error y hay que ajustar los pesos:

- Si  $\mathbf{w} \cdot \mathbf{x} < 0$  pero la salida debía ser  $y = 1$  — un **falso negativo**:
  - *Aumentar* los pesos de las entradas positivas.
  - *Disminuir* los pesos de las entradas negativas.
- Si  $\mathbf{w} \cdot \mathbf{x} > 0$  pero la salida debía ser  $y = 0$  — un **falso positivo**:
  - *Disminuir* los pesos de las entradas positivas.
  - *Aumentar* los pesos de las entradas negativas.

La **regla de aprendizaje del perceptrón** hace exactamente eso en una sola línea:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha (y - h_{\mathbf{w}}(\mathbf{x})) \mathbf{x}$$

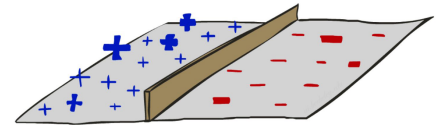
donde  $\alpha$  es la *tasa de aprendizaje* y el factor  $(y - h_{\mathbf{w}}(\mathbf{x}))$  vale  $+1$ ,  $-1$  o  $0$  (si no hay error, no hay actualización).

# Teorema de convergencia del perceptrón

Un problema de aprendizaje es **linealmente separable** si y solo si existe algún hiperplano que separa *exactamente* los ejemplos positivos de los negativos.

**Convergencia:** si los datos de entrenamiento son separables, aplicar repetidamente la regla de aprendizaje del perceptrón sobre el conjunto de entrenamiento converge, tarde o temprano, a un separador perfecto.

El problema es la letra pequeña: **si son separables**. La mayoría de problemas reales no lo son, y ahí el perceptrón nunca se estabiliza. Esa limitación es exactamente la que motiva el resto de la clase.



*Un caso **separable**: existe un hiperplano que deja todos los + de un lado y todos los - del otro.*

# La función de costo para clasificar: entropía cruzada

El perceptrón tiene modelo y regla de aprendizaje, pero le falta la pieza central de la receta: **una función de costo que podamos minimizar**. Su umbral duro responde 0 o 1, y eso no es derivable.

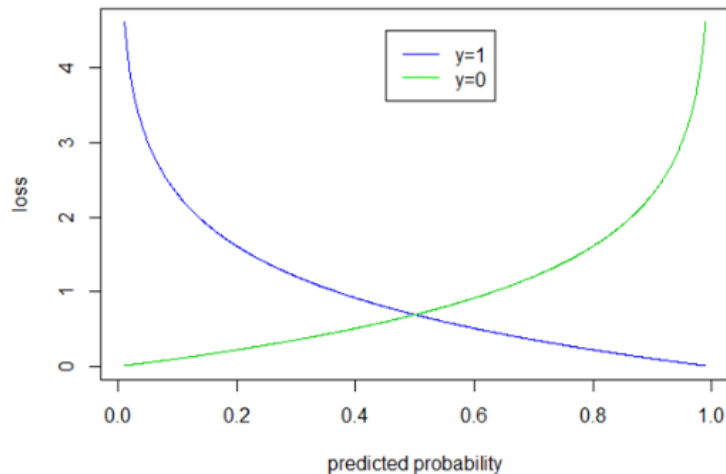
El arreglo es hacer que el clasificador devuelva una **probabilidad**  $h_{\theta}(\mathbf{x})$  entre 0 y 1 en lugar de una etiqueta —de eso se encarga la sigmoide, que veremos en la Parte 3—. Con una probabilidad a la salida sí podemos escribir un costo, y no es el error cuadrático: lo que queremos castigar no es la distancia sino la *confianza equivocada*. Esa función es la **entropía cruzada** (*cross-entropy*, o *log loss*):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$



# Entropía cruzada: cómo se comporta



Léanla curva por curva:

- **Azul ( $y = 1$ ):** si el modelo predice una probabilidad cercana a 1, el costo es casi 0; a medida que la probabilidad baja hacia 0, el costo **crece sin límite**.
- **Verde ( $y = 0$ ):** exactamente el espejo.
- Las dos se cruzan en  $p = 0.5$ , con costo  $\log 2 \approx 0.69$ : el precio de *no saber nada*.

La asíntota es lo importante: equivocarse **con confianza** cuesta infinito. Eso es lo que empuja al modelo no solo a acertar la clase, sino a estar seguro cuando acierta.

# Cross-Entropy Multiclase

## Cross-Entropy Multiclase

Compara la probabilidad asignada a la clase real

Fórmula general:

$$L = - \sum_{i=1}^C y_i \log(\hat{p}_i)$$

Con etiquetas one-hot (solo una clase real):

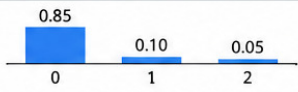
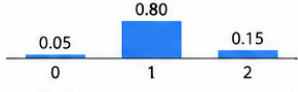
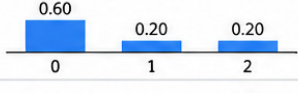
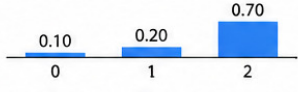
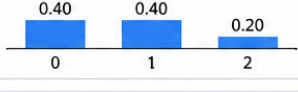
$$L = -\log(\hat{p}_{i_{\text{real}}})$$

$C$  : número de clases

$y_i$  : etiqueta real (one-hot)

$\hat{p}_i$  : probabilidad estimada por el modelo

$i_{\text{real}}$  : índice de la clase real

| Ejemplo                      | Etiqueta real (one-hot) | Probabilidades estimadas $\hat{p}$ (y gráfico)                                      | Probabilidad de la clase real $\hat{p}_{i_{\text{real}}}$ | Cálculo de la pérdida $L = -\log(\hat{p}_{i_{\text{real}}})$ | Interpretación                          |
|------------------------------|-------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------|
| <b>1</b><br>(clase real = 0) | [ 1 0 0 ]               |    | 0.85                                                      | $L = -\log(0.85) = 0.16$                                     | Baja pérdida<br>(predicción correcta)   |
|                              |                         |    | 0.10                                                      | $L = -\log(0.10) = 2.30$                                     | Alta pérdida<br>(predicción incorrecta) |
| <b>2</b><br>(clase real = 1) | [ 0 1 0 ]               |    | 0.80                                                      | $L = -\log(0.80) = 0.22$                                     | Baja pérdida<br>(predicción correcta)   |
|                              |                         |   | 0.20                                                      | $L = -\log(0.20) = 1.61$                                     | Alta pérdida<br>(predicción incorrecta) |
| <b>3</b><br>(clase real = 2) | [ 0 0 1 ]               |  | 0.70                                                      | $L = -\log(0.70) = 0.36$                                     | Baja pérdida<br>(predicción correcta)   |
|                              |                         |  | 0.20                                                      | $L = -\log(0.20) = 1.61$                                     | Alta pérdida<br>(predicción incorrecta) |

### Puntos clave:

1

La cross-entropy penaliza más cuando la probabilidad de la clase real es baja.

2

Con etiquetas one-hot, la pérdida depende solo de  $\hat{p}_{i_{\text{real}}}$ .

3

Cuanto mayor sea  $\hat{p}_{i_{\text{real}}}$ , menor será la pérdida.

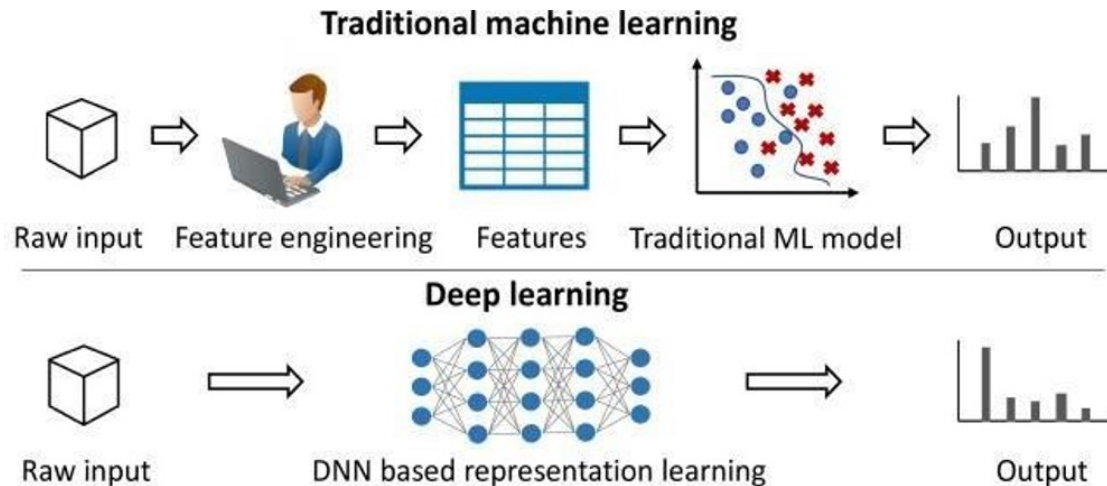
4

El modelo aprende a asignar alta probabilidad a la clase correcta para minimizar la cross-entropy.

# Parte 3 — Redes Neuronales y DNN

# Introducción: ¿cómo creamos las características?

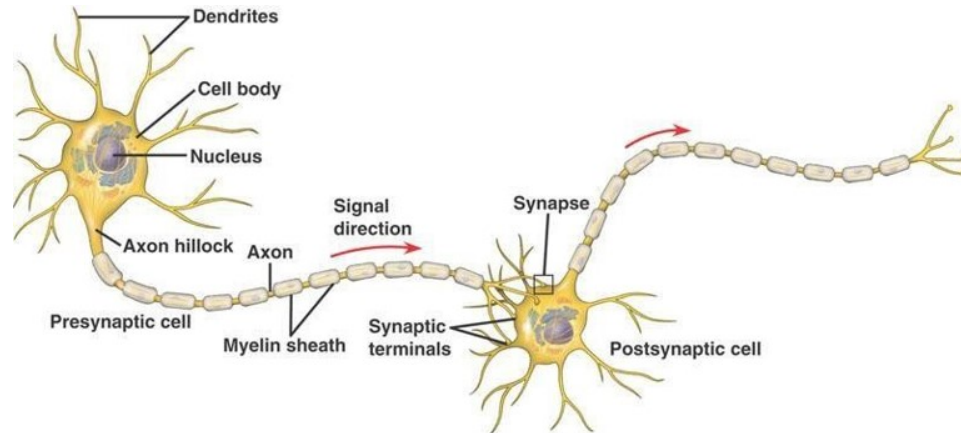
Todo lo anterior asume que **alguien ya definió las características**. En el aprendizaje de máquina tradicional ese alguien es una persona: es el trabajo de *ingeniería de características* (*feature engineering*), manual, costoso y dependiente del dominio.



La propuesta del **aprendizaje profundo** es eliminar ese paso: la red recibe la entrada cruda y **aprende ella misma la representación** que necesita para resolver la tarea.

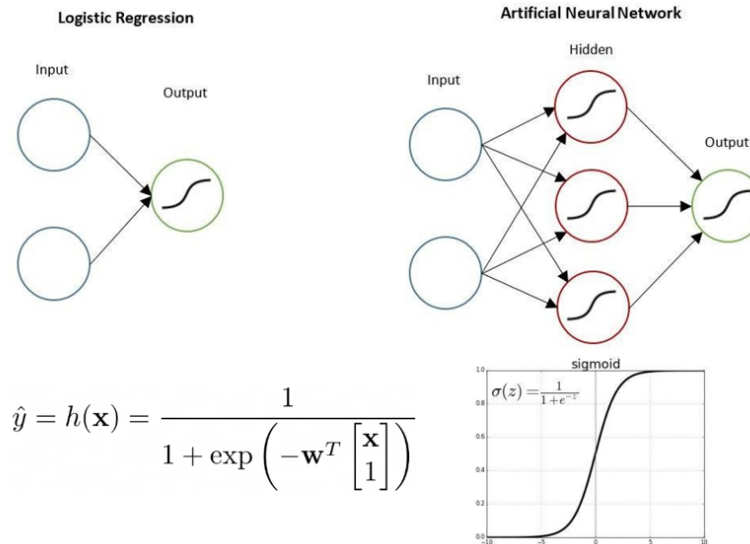
# Introducción

Las redes neuronales son modelos de aprendizaje de máquina inspirados en el cerebro humano. Aprenden patrones complejos a partir de los datos ajustando millones de parámetros distribuidos en capas interconectadas.



La analogía es directa: las *dendritas* reciben señales (las entradas), el *cuerpo celular* las integra (la suma ponderada), y el *axón* transmite el resultado a las neuronas siguientes (la salida).

# De la regresión logística a la red neuronal



A la izquierda, la **regresión logística**: una sola unidad que aplica la sigmoide al producto punto. A la derecha, una **red neuronal**: las mismas unidades, pero organizadas en una *capa oculta* cuya salida alimenta a la unidad final.

# Repaso: entradas, pesos y activación

- Las **entradas** son valores de características.
- Cada característica tiene un **peso**.
- La **suma** es la **activación**.

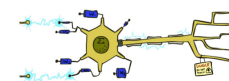
$$\text{activación}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

Y si la activación es:

- **Positiva**, la salida es  $+1$ .
- **Negativa**, la salida es  $-1$ .

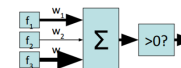
Es exactamente la misma unidad de la Parte 2 — solo que ahora la vamos a apilar.

- Inputs are **feature values**
- Each feature has a **weight**
- Sum is the **activation**



$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

- If the activation is:
  - Positive, output  $+1$
  - Negative, output  $-1$



# La función sigmoide

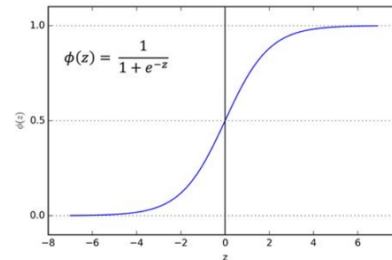
El umbral duro tiene un problema: es abrupto y no es derivable, así que no podemos entrenarlo con gradientes. Lo reemplazamos por una transición **suave**.

- Activación:  $z = w \cdot f(x)$ .
- Si  $z$  es muy **positiva** → queremos una probabilidad que tienda a **1**.
- Si  $z$  es muy **negativa** → queremos una probabilidad que tienda a **0**.

- Activation:  $z = w \cdot f(x)$
- If  $z$  is very positive □ want probability going to 1
- If  $z$  is very negative □ want probability going to 0

- Sigmoid function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



$$\phi(z) = \frac{1}{1 + e^{-z}}$$



# Clasificación lineal multiclase

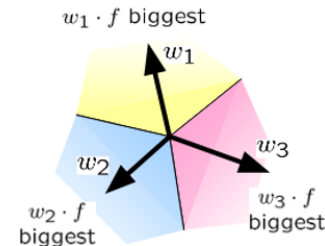
¿Y si hay más de dos clases? Ya no basta un hiperplano: usamos

un vector de pesos por clase

y nos quedamos con el de mayor puntaje.

- Multi-class linear classification

- A weight vector for each class:  $w_y$
- Score (activation) of a class  $y$ :  $w_y \cdot f(x)$
- Prediction w/highest score wins:  $y = \arg \max_y w_y \cdot f(x)$



- How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

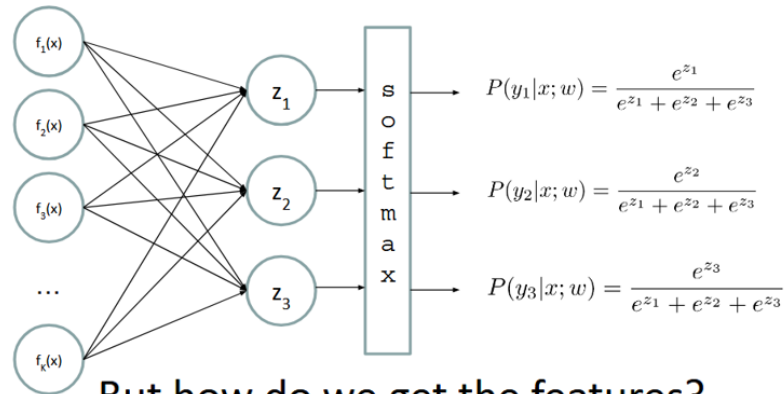
---

class: smaller

- Un vector de pesos para cada clase:  $w_y$ .

# Softmax

$$h_{\theta}(x) = \begin{bmatrix} P(y=1|x;\theta) \\ P(y=2|x;\theta) \\ \vdots \\ P(y=K|x;\theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^K \exp(\theta^{(j)\top} x)} \begin{bmatrix} \exp(\theta^{(1)\top} x) \\ \exp(\theta^{(2)\top} x) \\ \vdots \\ \exp(\theta^{(K)\top} x) \end{bmatrix}$$



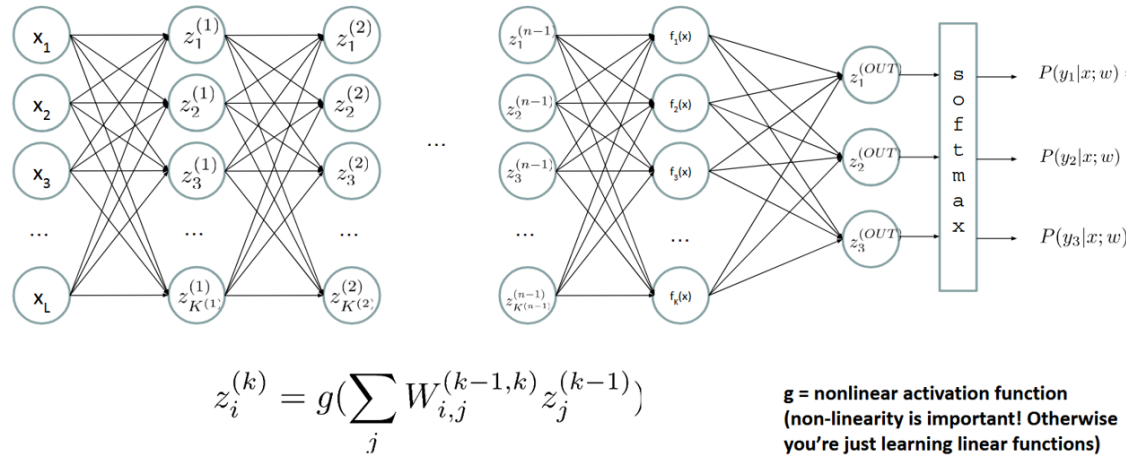
But how do we get the features?

Softmax convierte  $K$  puntajes arbitrarios en una distribución de probabilidad válida (todos positivos y que suman 1).

Pero seguimos con la misma pregunta abierta: ¿de dónde salen las características  $f(x)$ ?

# Deep Neural Network (DNN)

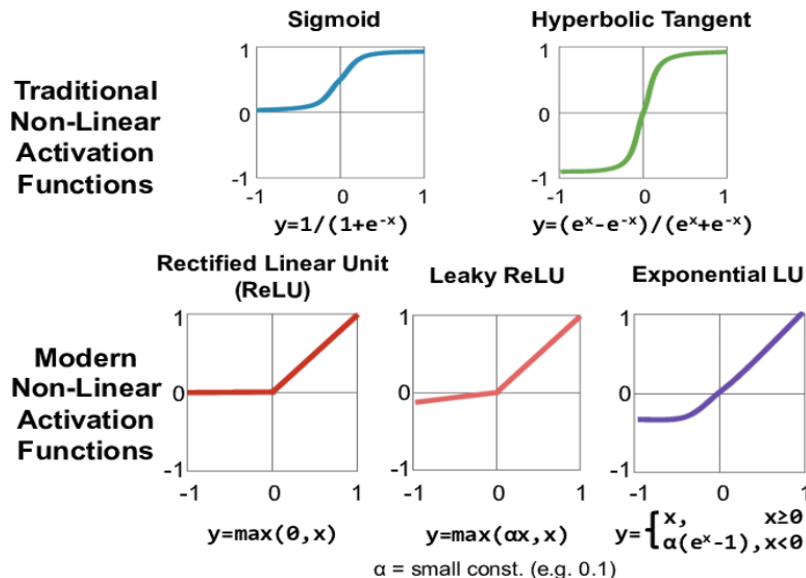
La respuesta: apilar capas. Cada capa toma la salida de la anterior, aplica una transformación lineal y luego una **no linealidad**.



$$z_i^{(k)} = g \left( \sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)} \right)$$

$g$  es una **función de activación no lineal** — y la no linealidad es indispensable: sin ella, componer capas lineales da otra función lineal, y toda la profundidad no serviría de nada.

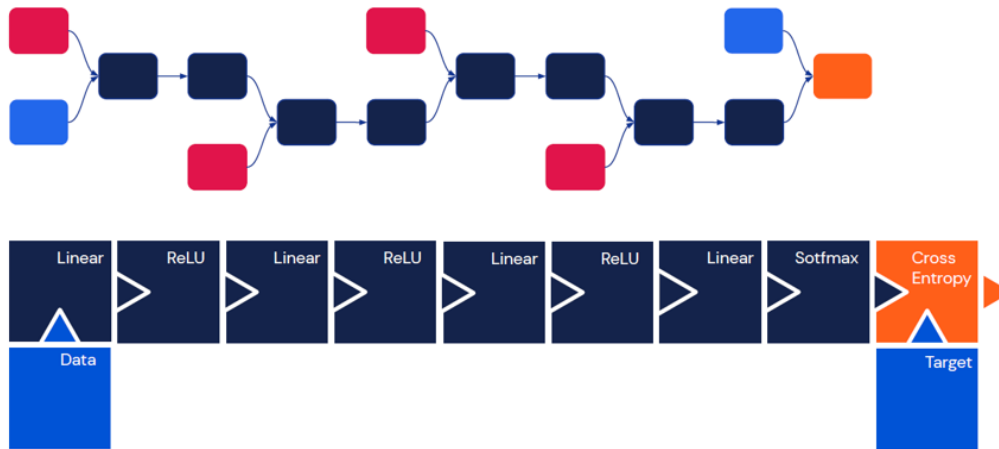
# Funciones de activación



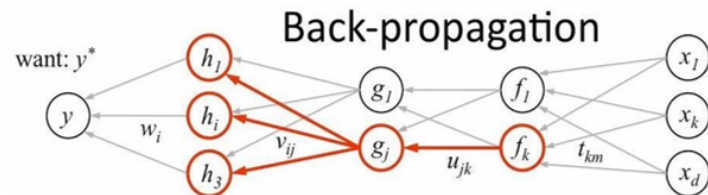
**Tradicionales:** sigmoide y tangente hiperbólica — saturan en los extremos, lo que apaga el gradiente en redes profundas. **Modernas:** ReLU y sus variantes (Leaky ReLU, ELU) — baratas de calcular y sin saturación del lado positivo.

*ReLU es hoy la opción por defecto.*

# Entrenamiento: backpropagation



1. receive new observation  $\mathbf{x} = [x_1 \dots x_d]$  and target  $y^*$
2. **feed forward:** for each unit  $g_j$  in each layer  $1 \dots L$  compute  $g_j$  based on units  $f_k$  from previous layer:  $g_j = \sigma \left( u_{j0} + \sum_k u_{jk} f_k \right)$
3. get prediction  $y$  and error  $(y - y^*)$
4. **back-propagate error:** for each unit  $g_j$  in each layer  $L \dots 1$



# El gradiente

## El gradiente: ¿hacia dónde mover la perilla?

El gradiente indica la dirección y qué tan grande debe ser el cambio en cada parámetro.

### Gradiente positivo ( $\frac{\partial L}{\partial w} > 0$ )



Para disminuir la pérdida, mueve la perilla hacia la izquierda ( $w \downarrow$ ).

### Gradiente negativo ( $\frac{\partial L}{\partial w} < 0$ )



Para disminuir la pérdida, mueve la perilla hacia la derecha ( $w \uparrow$ ).

### Gradiente cero ( $\frac{\partial L}{\partial w} = 0$ )



No hay dirección preferida (ya estamos en un punto estacionario).

### Actualización de un parámetro

$$w_{\text{nuevo}} = w_{\text{actual}} - \eta \frac{\partial L}{\partial w}$$

- $\eta$  : tasa de aprendizaje
- $\frac{\partial L}{\partial w}$  : gradiente (en  $w_{\text{actual}}$ )

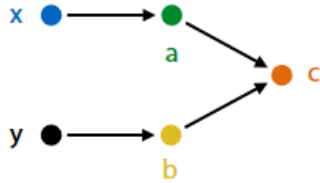
### Con muchos parámetros

$$\nabla L(w) = \left[ \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right]$$

El gradiente nos da una **instrucción** (dirección y tamaño) para cada perilla del modelo.

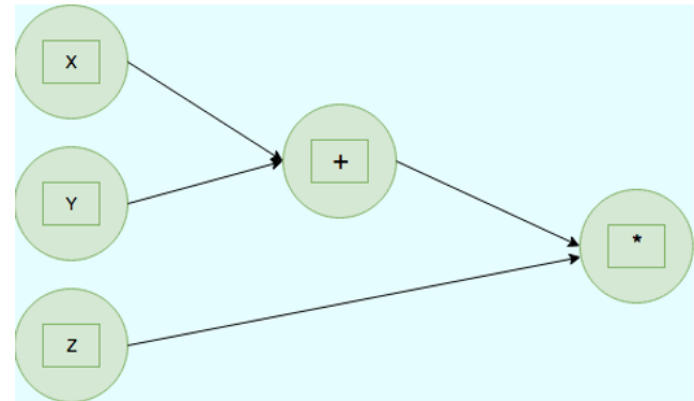
**En resumen: el gradiente es la brújula que nos dice cómo ajustar cada parámetro para disminuir la pérdida.**

# Backpropagation es la regla de la cadena



$$\frac{\partial c}{\partial x} = \frac{\partial c}{\partial a} \frac{\partial a}{\partial x} \quad \frac{\partial c}{\partial y} = \frac{\partial c}{\partial b} \frac{\partial b}{\partial y}$$

*Cada arista es una derivada parcial local.*



*Toda expresión —y toda red— es un grafo de operaciones elementales.*

Backpropagation = recorrer ese grafo **de la salida hacia la entrada** multiplicando derivadas locales. Ni más, ni menos.

1. Recibir una nueva observación  $\mathbf{x} = [x_1 \dots x_d]$  y su objetivo  $y^*$ .
2. **Propagación hacia adelante** (*feed forward*): para cada unidad  $g_j$  en cada capa  $1 \dots L$ , calcular  $g_j$  a partir de las unidades  $f_k$  de la capa anterior.
3. Obtener la predicción  $y$  y el error  $(y - y^*)$ .
4. **Propagar el error hacia atrás** (*back-propagate*): para cada unidad  $g_j$ , desde la capa  $L$  hasta la  $1$ , repartir la responsabilidad del error y actualizar los pesos.

### Gradient

$$y = f(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}$$

$$\frac{\partial y}{\partial \mathbf{x}} = \nabla_{\mathbf{x}} f(\mathbf{x}) = \left[ \frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_d} \right]$$

### Jacobian

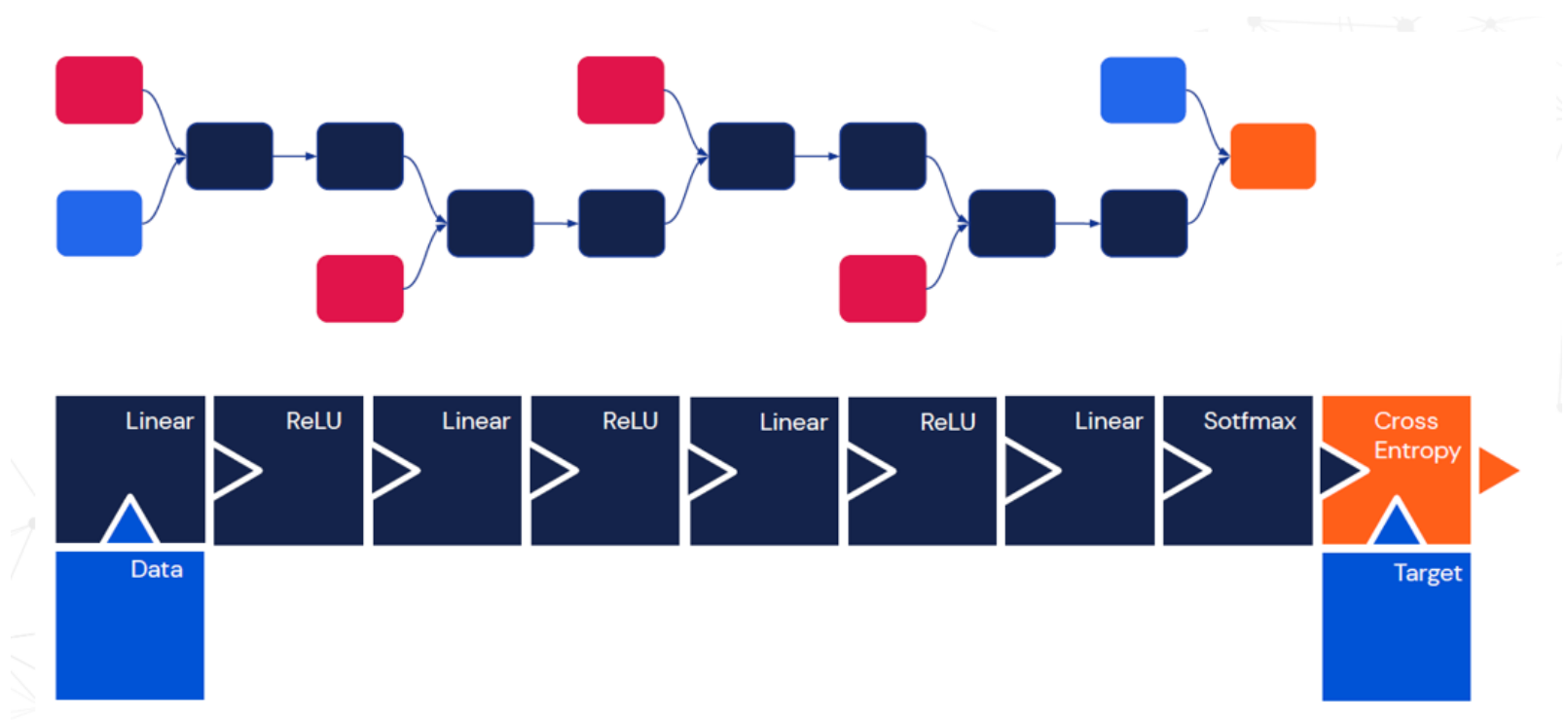
$$\mathbf{y} = f(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^k$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{J}_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_k}{\partial x_1} & \dots & \frac{\partial f_k}{\partial x_d} \end{bmatrix}$$

- Al final del grafo está el **gradiente**: la función de costo devuelve un **escalar**, y su derivada respecto a las entradas es un **vector**.
- En cada capa intermedia está el **jacobiano**: la capa transforma un vector en otro vector, y su derivada es una **matriz**.
- Propagar el error hacia atrás es **encadenar esas matrices**, capa por capa, hasta llegar a los pesos.

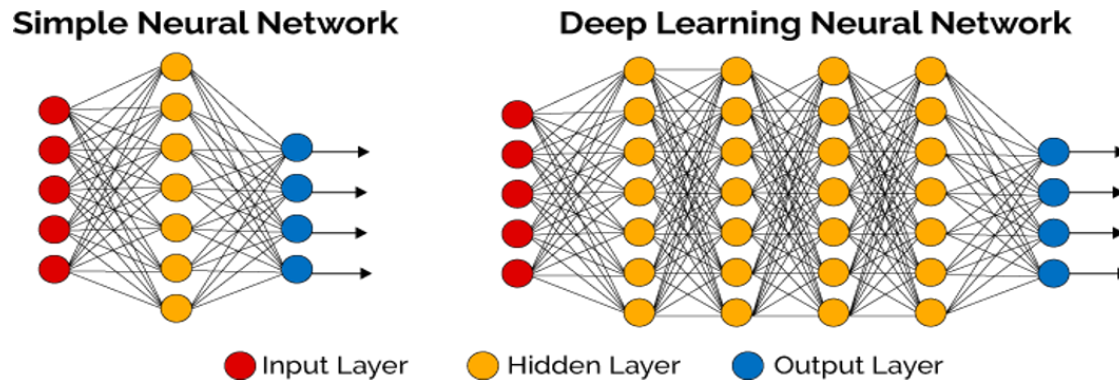


# La red completa, vista como un grafo



Así es como una red profunda existe realmente: una cadena de bloques —**Linear**, **ReLU**, **Linear**, ..., **Softmax**— que termina en la **entropía cruzada**.

# Red neuronal simple vs. red profunda



La diferencia entre una y otra es el **número de capas ocultas**. Cada capa adicional construye representaciones sobre las de la capa anterior: las primeras capas capturan patrones simples, las últimas capturan conceptos cada vez más abstractos.

# Aproximadores universales y consideraciones prácticas

- Theorem (Universal Function Approximators). A two-layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.
- Practical considerations
  - Can be seen as learning the features
  - Large number of neurons
    - Danger for overfitting
    - (hence early stopping!)

# Ejemplo y ejercicio de clase

Class example and exercise



Las dos bibliotecas estándar para construir y entrenar redes neuronales. Ambas implementan por nosotros la diferenciación automática — es decir, backpropagation.

# Resumen

- Todo aprendizaje supervisado es la misma receta: un **modelo**  $f$ , una **función de costo**  $\mathcal{L}$ , y un procedimiento que ajusta los parámetros para minimizarla.
- La **regresión lineal** es el caso más simple:  $\hat{y} = \mathbf{w}^T \mathbf{x}$ , costo cuadrático medio, y dos formas de resolverla — descenso por gradiente o ecuación normal.
- El **clasificador lineal** reutiliza el mismo producto punto pero le aplica un *umbral*: define un hiperplano que separa el espacio en dos clases.
- La **regla del perceptrón** converge a un separador perfecto *solo si* los datos son linealmente separables — una limitación seria en la práctica.
- Para clasificar, la función de costo es la **entropía cruzada**: castiga sin límite la confianza equivocada, y su gradiente conserva la forma *error*  $\times$  *entrada*.
- La **sigmoide** suaviza el umbral (da probabilidades y es derivable) y **softmax** la generaliza a  $K$  clases.
- Una **red neuronal** apila estas unidades en capas; las capas ocultas *aprenden las características* que antes había que diseñar a mano.
- La **no linealidad**  $g$  es indispensable: sin ella, apilar capas lineales sigue dando una función lineal.
- **Backpropagation** es la regla de la cadena aplicada eficientemente sobre el *grafo de cómputo* de la red: se encadenan los jacobianos de cada capa desde la salida hacia la entrada. Quien entrena sigue siendo el descenso por gradiente.
- Con suficientes neuronas, una red de dos capas es un **aproximador universal** — con el riesgo asociado de *sobreajuste*.

# Fin de la Clase 7

Próxima clase: Redes Neuronales Convolucionales (CNN)